

GamutX

Operators, Strings, & Files

Python Fundamentals

What Are Unary Operators?

- What Are Unary Operators?
- Types of Unary Operators
- Unary Plus (+)
- Unary Minus (-)
- Bitwise NOT (~)

What Are Unary Operators?

- Operators are special symbols used to perform operations on values or variables
- Unary operators **work with only one operand**
- They are called unary because **“uni” means one**
- The operation is applied to a **single value or variable**

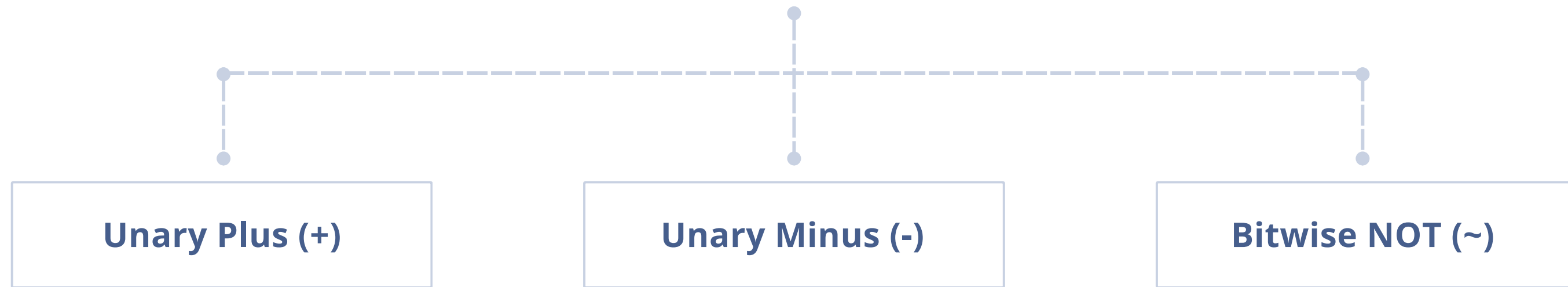
Example :



Types of Unary Operators

GamutX

Python mainly supports
Three unary operators



Unary Plus (+)

- Used to indicate a **positive number**.
- It doesn't change the value, just represents it as positive.

```
a = 10
b = +a -----> indicate a positive number

print("a =", a)    # 10
print("b =", b)    # 10
```

Unary Minus (-)

- Used to **negate a value**.
- Converts positive → negative and negative → positive.

```
x = 7
y = -x -----> changes sign

print("x =", x)    # 7
print("y =", y)    # -7

z = -(-10) -----> double negation

print("z =", z)    # 10
```

Bitwise NOT (~)

- The bitwise NOT operator works on the **binary representation of an integer**
- It **flips every bit**:
 - `1 → 0`
 - `0 → 1`
- **Formula:** `~n = -(n + 1)`

Positive Number

```
p = 5
q = ~p

print("p =", p)    # 5
print("q =", q)    # -6
```

Negative Number

```
n = -10
print(~n)          # 9
```

BTS:

```
~(-10) = -((-10) + 1)
        = -(-9)
        = 9
```

Summary

Operator		Meaning		Example		Result
<code>+X</code>	→	Unary Plus (no change)	→	<code>+7</code>	→	<code>7</code>
<code>-X</code>	→	Unary Minus (negation)	→	<code>-7</code>	→	<code>-7</code>
<code>~X</code>	→	Bitwise NOT (flip bits)	→	<code>~5</code>	→	<code>-6</code>

Assignment Operators

What is an Assignment Operator

Types of Assignment Operators

Simple Assignment

Add & Assign ($+=$)

Subtract & Assign ($-=$)

Multiply & Assign ($*=$)

Divide & Assign ($/=$)

Floor Divide & Assign ($//=$)

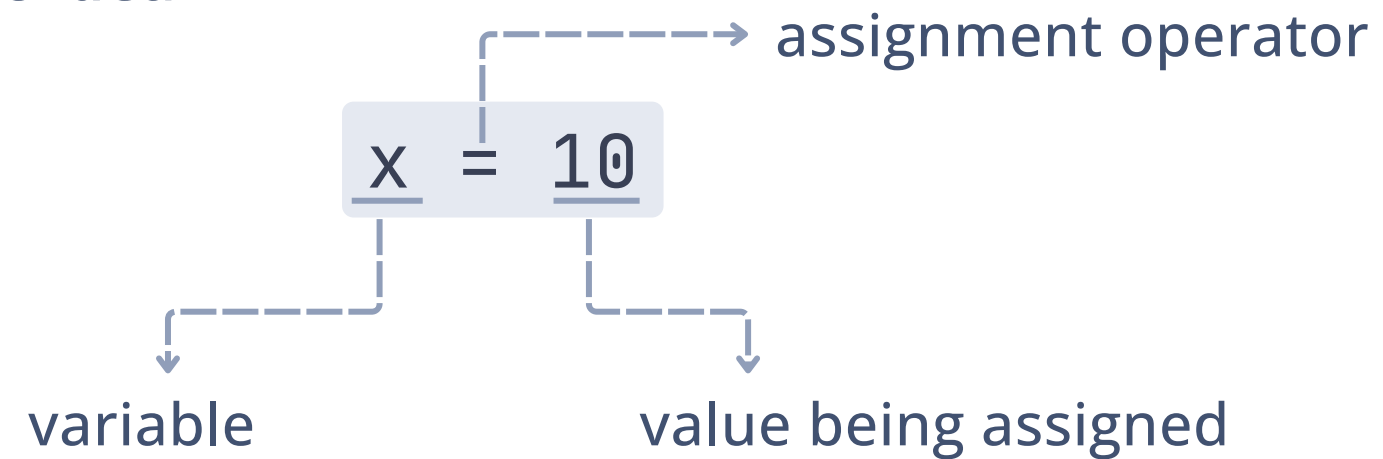
Modulus & Assign ($\%=$)

Exponentiation & Assign ($**=$)

What is an Assignment Operator

- Assignment operators are **used to store values in variables**
- They can also be used to **update existing values**
- They connect a **value** or **expression to a variable**
- They are a **basic building block** of programming
- Without assignment operators, programs **cannot remember or change data**

Basic Idea →



Types of Assignment Operators

=	→	Simple Assignment
+=	→	Add and Assign
-=	→	Subtract and Assign
*=	→	Multiply and Assign
/=	→	Divide and Assign
%=	→	Modulus and Assign
**=	→	Exponent and Assign
//=	→	Floor Divide and Assign

Simple Assignment

- **Purpose:** Assigns a value to a variable.
- **Syntax:** `variable = value`

Example

```
frame = 100
```

Assignment Operator

```
print(frame) # Output: 100
```

- **Use case:** Anytime you need to **store a value** in a variable.

Add & Assign (+=)

- **Purpose:** Adds a value to a variable and updates it.
- **Syntax:** `variable += value`

Example

```
frame = 100  
frame += 10  
print(frame) # Output: 110
```

-----> Frame = Frame + 10

Explanation:

- `frame += 10` adds 10 to the current value of frame (100 → 110).
- Saves writing the variable name twice.

Subtract & Assign (-=)

- **Purpose:** Subtracts a value from a variable and **updates it**.
- **Syntax:** `variable -= value`

Example

```
opacity = 1.0  
opacity -= 0.2  
  
print(opacity) # Output: 0.8
```

→ $\text{Opacity} = \text{Opacity} - 0.2$

- Useful for gradual reductions, like fading out opacity in animations or effects.

Multiply & Assign (*=)

- **Purpose:** Multiplies a variable by a value and **updates it**.
- **Syntax:** `variable *= value`

Example

```
scale = 2  
scale *= 3  
  
print(scale) # Output: 6
```

-----> Scale = Scale * 3

- Useful in scaling objects, adjusting size, or mathematical calculations.

Divide & Assign (/=)

- **Purpose:** Divides a variable by a value and **updates it**.
- **Syntax:** `variable /= value`

Example

```
duration = 60
duration /= 4
print(duration) # Output: 15.0
```

→ duration = duration / 4

Explanation:

- Produces a float result, even if both operands are integers.
- Useful for splitting durations, timings, or averaging values.

Floor Divide & Assign (//=)

- **Purpose:** Divides a variable by a value, **keeps only the integer result**, and updates it.
- **Syntax:** `variable //= value`

Example

```
frames = 100  
frames //= 6  
  
print(frames) # Output: 16
```

-----> frame = frame // 6

Useful in frame batching, loops, or any scenario where only whole numbers matter.

Modulus & Assign (%=)

- **Purpose:** Takes the remainder of division and updates the variable
- **Syntax:** `variable %= value`

Example

```
frame = 37  
frame %= 10  
  
print(frame)  # Output: 7
```

-----> frame = frame % 10

- Useful for looping through frame ranges, cycling values, or repeating patterns.

Exponentiation & Assign (**=)

- **Purpose:** Raises a variable to a power and updates it.
- **Syntax:** `variable **= value`

Example

```
size = 5  
size **= 2 -----> size = (size)2  
  
print(size) # Output: 25
```

- Useful in scaling effects, resolution multipliers, and calculations involving powers.

Membership Operators

- What is a Membership Operators
- Using in
- Using not in

What is a Membership Operators

- Membership operators are used to **test whether a value exists in a sequence** .
- They **return True or False**.

Types of Membership Operators

Operator	Meaning	Example
• <code>in</code>	→ Returns True if a value exists in a sequence	→ <code>x in list</code>
• <code>not in</code>	→ Returns True if a value does NOT exist in a sequence	→ <code>x not in list</code>

Works with **strings, lists, tuples, sets, and dictionaries (keys)**.

Using in

```
fruits = ['apple', 'banana', 'cherry']  
  
print('apple' in fruits)    # True  
  
print('orange' in fruits)    # False
```

Using not in

```
numbers = [1, 2, 3, 4, 5]  
  
print(6 not in numbers)    # True  
  
print(3 not in numbers)    # False
```

6 is not in the list

not in is just the **inverse of in**.

With Strings

```
text = "Hello, World!"  
print("Hello" in text)      # True  
print("Python" not in text) # True
```

"Hello" exists in "Hello, World!"

"Python" does not exist

Key Points

- in → checks for presence.
- not in → checks for absence.
- Can be used in conditionals and loops.
- Works with most iterable types.

Identity Operators

- What is a Identity Operators
- Using is
- Using is not
- Common Use Case

What is a Identity Operators

- Identity operators are used to **check whether two variables refer to the same object in memory,**
- not just whether their values are equal.

Types of Identity Operators

Operator	Meaning
• <code>is</code>	→ Returns True if two variables refer to the same object
• <code>is not</code>	→ Returns True if two variables refer to different objects

Using is

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
```

```
print(a is b)
# True
```

point to the same
memory location.

```
print(a is c)
# False
```

```
print(a == c)
# True
```

compare values

Using is not

```
x = 10
y = 20
```

different objects

```
print(x is not y) # True
```

is not returns **True** when two variables are **not the same object** in memory.

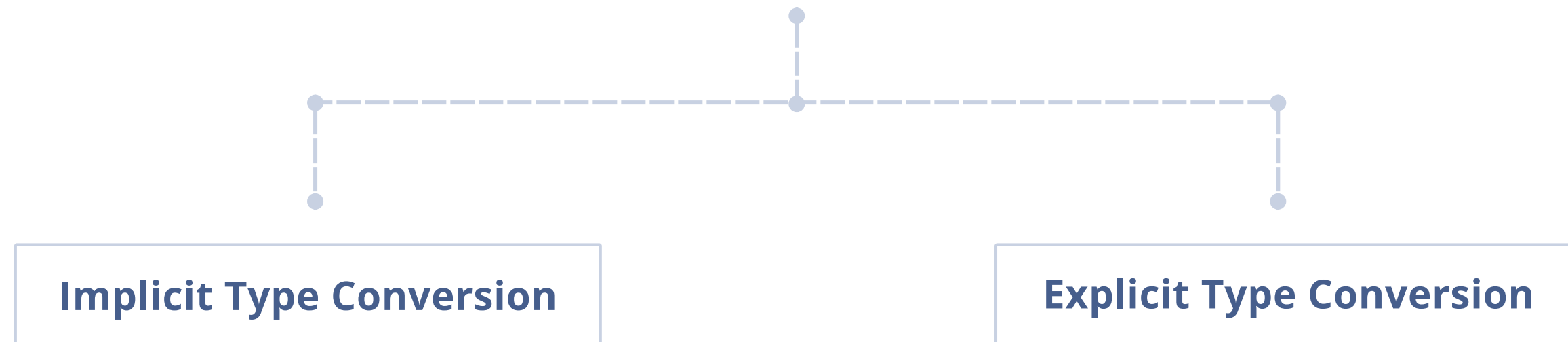
Type Conversion

- What is Type Conversion
- Implicit Type Conversion
- Explicit Type Conversion

What is Type Conversion

- Type conversion is the process of **changing the data type of a value from one type to another.**
- It is also known as **type casting.**

Two Types of Type Conversion:



Implicit Type Conversion

- Done automatically by Python
- Python **converts a smaller data type into a larger data type**
- This helps prevent data loss

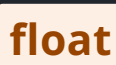
Adding Int and Float

```
a = 5 --> int  
b = 2.5 --> float  
result = a + b  
  
print(result)           # 7.5  
print(type(result))    # <class 'float'>
```

Boolean in Arithmetic

```
a = True --> Boolean (internally 1)  
b = 5 --> int  
  
print(a + b)           # 6  
print(type(a + b))    # <class 'int'>
```

Multiplying Int and Float

```
a = 10    
b = 3.2    
c = a * b  
  
print(c)          # 32.0  
print(type(c))    # <class 'float'>
```

Key Points

- Happens **automatically** — you don't write conversion code.
- Only **safe conversions** are allowed.
- Most common case: **int** → **float** in arithmetic operations.
- Ensures **no information is lost** during operations

Explicit Type Conversion

- **Explicit conversion** = manually change the **type** of a value using built-in functions.
- This is also called **type casting**.
- Unlike implicit conversion (automatic), here you **decide** and **control** the type conversion.

Common Conversions:

• `int(x)` → convert to integer

• `float(x)` → convert to float

• `str(x)` → convert to string

• `list(x)` → convert to list

• `tuple(x)` → convert to tuple

Converting Float to Int

```
value = 9.7
converted = int(value)

print(converted)      # 9
print(type(converted)) # <class 'int'>
```

⚠ Note:

- `int()` **removes the decimal part**, it does not round.

Converting Number to String

```
frame = 100
label = "Frame_" + str(frame)

print(label) # "Frame_100"
```

Example:

- Converts numbers to string to concatenate with text.
- Useful in file naming, labels, and logging

Converting Int to Float

```
frames = 120
seconds = float(frames) / 24

print(seconds) # 5.0
```

Example:

- Converts integer frames to float for decimal calculations.
- Common when calculating time from frame numbers or percentages.

Converting String to Int/Float

```
fps = "24"  
fps_int = int(fps)  
  
print(fps_int * 2)  # 48
```

Example:

- Converts numeric strings into actual numbers.
- Useful when reading input, configuration files,

Boolean Conversion

```
print(bool(0))      # False  
print(bool(5))      # True  
print(bool(""))     # False  
print(bool("vfx"))  # True
```

Example:

- Converts values to True or False.
- Handy for on/off toggles in scripts or conditional checks.

Key Takeaway

- Explicit conversion = you control the conversion.
- Common functions: `int()`, `float()`, `str()`, `bool()`, `list()`, `tuple()`, `set()`.
- Necessary when working with:
 - User input
 - File data
 - Mixed data types
 - Text and number concatenation

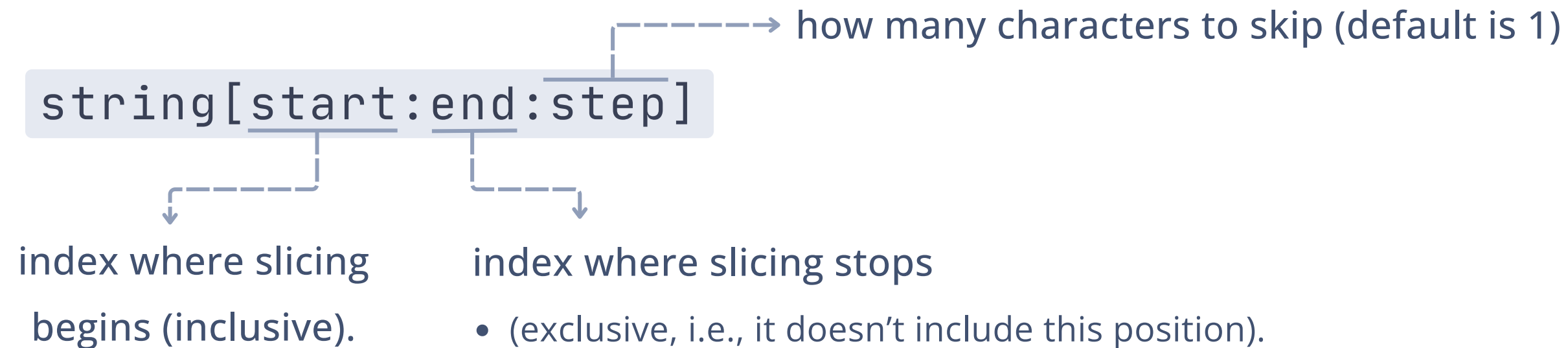
String Slicing

- What are String Slicing ?
- Basic slicing
- Omit start
- Omit end
- Slicing with step
- Reverse String
- Negative indexing

What are String Slicing ?

String slicing is a **way to cut out a portion of a string by specifying start, end, and an optional step.**

Basic Syntax →



Basic slicing

Basic slicing is used to cut out a part of a sequence by specifying the start and end indices.

```
s = "Python"
print(s[0:4]) # Output: "Pyth"
```

n-1

Slice from index 0 to 3

Visual:

P	y	t	h	o	n
0	1	2	3	4	5

Explanation:

- Syntax: s[start:end]
- start = 0 → begin at index 0 (P)
- end = 4 → stop before index 4 (o)
- Extracted characters = index 0,1,2,3 → "Pyth"

Omit start

Omitting the start index makes **slicing begin from the start of the sequence (index 0)**.

```
s = "Python"
print(s[:4])    # Output: "Pyth"
```

Start index omitted, defaults to 0

Visual:

P	y	t	h	o	n
0	1	2	3	4	5

Explanation:

- start is omitted → defaults to 0
- end = 4 → stop before index 4
- Result = "Pyth" (same as previous example)

Omit end

Omitting the end index **makes slicing go all the way to the last element** of the sequence.

```
s = "Python"
print(s[2:])  # Output: "thon"
```



Visual:

P	y	t	h	o	n
0	1	2	3	4	5

s[2:]

Explanation:

- start = 2 → begin at index 2 (t)
- end is omitted → defaults to end of string (len(s) = 6)
- Extracted characters = index 2,3,4,5 → "thon"

Slicing with step

Slicing can also include a **step**, which lets you **skip elements at regular intervals**.

```
s = "Python"
print(s[0:6:2]) # Output: "Pto"
```

start

stop

step

Visual:

P	y	t	h	o	n
0	1	2	3	4	5

Explanation:

- Syntax: `s[start:end:step]`
- start = 0, end = 6 (up to index 5)
- step = 2 → take every 2nd character
- Extracted characters: index 0 → P, index 2 → t, index 4 → o
- Result = "Pto"

Reverse String

Reversing a string can be **done with slicing by omitting start and end and using a step of -1.**

```
s = "Python"
print(s[::-1]) # Output: "nohtyP"
```

Move Backward one char at a time

Explanation:

- start and end are omitted → default to full string
- step = -1 → move backwards one character at a time
- Characters extracted: n, o, h, t, y, P → "nohtyP"

Negative indexing

- Negative indexing lets you **access elements from the end of a sequence**,
- with -1 being the last element, -2 the second last, and so on.

```
s = "Python"
print(s[-4:-1]) # Output: "tho"
```



start at -4 end before -1

Visual:

s = "Python"

----->

-6	-5	-4	-3	-2	-1	→ Index's
p	y	t	h	o	n	→ Chars

String 'is' Method

- What are is methods?
- Common is Methods

What are 'is' methods?

- is method are used to **check properties of a string**.
- All is methods:
 - Start with the **prefix is**
 - Work **only on strings**
 - **Return a Boolean** value:
 - True
 - False
- They do not change the original string

Syntax →

```
string.is_property()
```

the string we want to check

the condition we want to test

Common is Methods

GamutX

Method		Description
<code>isalpha()</code>	→	Returns True if all characters are letters (A–Z, a–z)
<code>isdigit()</code>	→	Returns True if all characters are digits (0–9)
<code>isalnum()</code>	→	Returns True if all characters are letters or digits
<code>isspace()</code>	→	Returns True if all characters are whitespace
<code>isupper()</code>	→	Returns True if all letters are uppercase
<code>islower()</code>	→	Returns True if all letters are lowercase
<code>istitle()</code>	→	Returns True if string is title-cased (first letter uppercase, rest lowercase in each word)

isalpha()

Returns True if all characters are letters (A–Z, a–z)

```
word1 = "Python"
word2 = "Py123"

print(word1.isalpha()) # True
print(word2.isalpha()) # False
```

isalnum()

Returns True if all characters are letters or digits

```
mix1 = "Hello123"
mix2 = "   "

print(mix1.isalnum()) # True
print(mix2.isalnum()) # False
```

isdigit()

Returns True if all characters are digits (0–9)

```
num1 = "12345"
num2 = "123abc"

print(num1.isdigit()) # True
print(num2.isdigit()) # False
```

isspace()

Returns True if all characters are whitespace

```
space1 = "   "
space2 = "Python"

print(space1.isspace()) # True
print(space2.isspace()) # False
```

isupper()

Returns True if all letters are uppercase

```
upper1 = "HELLO"  
upper2 = "Hello"  
  
print(upper1.isupper()) # True  
print(upper2.isupper()) # False
```

islower()

Returns True if all letters are lowercase

```
lower1 = "hello"  
lower2 = "Hello"  
  
print(lower1.islower()) # True  
print(lower2.islower()) # False
```

istitle()

Returns True if string is title-cased

```
title1 = "Hello World"  
title2 = "Python"  
title3 = "hello"  
  
print(title1.istitle()) # True  
print(title2.istitle()) # True  
print(title3.istitle()) # False
```

String Alignment & Padding

- What is String Alignment & Padding?
- Left Alignment
- Right Alignment
- Center Alignment
- Zero Padding

What is String Alignment & Padding?

- **String alignment** → means **controlling how text is positioned within a given width.**
- **Padding** → is the process of **filling extra space with characters** (like spaces or zeros) to **reach that width.**

Built-in string methods for alignment and padding:

- • `str.ljust(width, fillchar)`
- • `str.rjust(width, fillchar)`
- • `str.center(width, fillchar)`
- • `str.zfill(width)`

Left Alignment

Left-align → string starts at the left, padding added to the right

```
s = "Python"
```

```
print(s.ljust(10, '-')) # Python----
```

P y t h o n - - -
6 4

Left-aligns text

Total width of final string

Padding character

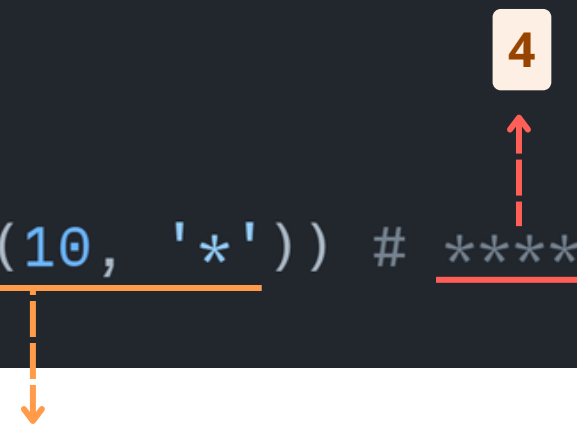
Explanation:

- width = 10 → total length of output
- Original string = 6 characters → 4 extra spaces filled with '-'

Right Alignment

String is right-aligned, padding added to the left

```
s = "Python"
print(s.rjust(10, '*')) # ****Python
```



Right-aligns text within given width

Center Alignment

String is centered, padding distributed evenly on both sides

```
s = "Python"
print(s.center(12, '.')) # ...Python...
```



Centers text within given width

Zero Padding

Pads numeric string with **zeros** on the left . Works only for **numeric strings**

```
num = "42"  
  
print(num.zfill(5))  # 00042
```

Pads string with leading zeros

Total width after fill

File Handling Modes

- Revision of File Modes
- Read & Write (r+)
- Write & Read (w+)
- Append & Read (a+)

Revision of File Modes

Allows you to **create, read, write, and manipulate files** (like .txt, .csv, etc.) on your computer.

◆ "r" (Read Mode)

- Opens a file for **reading only**.
- File must exist, otherwise it throws an error.

```
f = open("file.txt", "r")
```

Opens file for reading only

◆ "w" (Write Mode)

- Opens a file for **writing**.
- Creates the file if it does not exist.

```
f = open("file.txt", "w")
```

Opens file for writing, overwrites existing

◆ "a" (Append Mode)

- Opens a file for **adding new data at the end**.
- Creates the file if it does not exist.

```
f = open("file.txt", "a")
```

append at end

Read & Write (r+)

Open an **existing file** for **reading and writing**. Will **raise an error** if the file does **not exist**.

```
with open("demo_read.txt", "w") as f:
    f.write("Python")

with open("demo_read.txt", "r+") as f:
    print("Before:", f.read())

    f.seek(0)
    f.write("Hello")
    f.seek(0)

    print("After:", f.read())
```

Annotations:

- `f.read()` → read existing content
- `f.seek(0)` → move cursor to start
- `f.write("Hello")` → overwrite from start
- `f.read()` → read updated content

Before: Python
After: Hellon

Before Read Write

```
demo_read.txt
1 Python
```

After Read Write

```
demo_read.txt
1 Hellon
```

Write & Read (w+)

- Open a file for **writing and reading**. Creates the file if it **does not exist**.
- If the file exists, **old content is cleared**.

```
with open("demo_write.txt", "w+") as f:  
    f.write("First Content")  
    f.seek(0)  
    print("Reading:", f.read())
```

writes data to the file

moves the cursor back to the start

```
# Reading: First Content
```

Output

```
demo_write.txt  
1 First Content  
2
```

Behavior:

- Cursor starts at the beginning.
- Can read what you've just written using `f.seek(0)`.

Append & Read (a+)

- Open a file for **appending and reading**. Creates the file if it **does not exist**.
- Keeps old content; new data is **always added at the end**.

```
with open("demo_append.txt", "w") as f:  
    f.write("Line 1\n")
```

```
with open("demo_append.txt", "a+") as f:  
    f.write("Line 2\n") → append
```

```
f.seek(0) → go back to start for reading  
print("Reading:\n", f.read())
```

```
# Reading:  
# Line 1  
# Line 2
```

Output

```
demo_a+.txt  
1 Line 1  
2
```

Output after Append & Read

```
demo_a+.txt  
1 Line 1  
2 Line 2  
3
```

Behavior:

- Writing always appends.
- To read, move cursor to start with `f.seek(0)`.

When to Use

◆ r+ - Read + Write

- **File must exist** and you **want to read it and update specific parts** (without clearing it).

◆ w+ - Write + Read

- You want a **fresh file, discard old content, write new data**, then **read it**.

◆ a+ - Append + Read

- You want to **keep existing data** and only **add new content at the end**.

Understanding 'x' Mode

- What is 'x' Mode?
- Using 'x' Mode
- Why Use 'x' Mode?

What is 'x' Mode?

- In Python, '**x**' **mode** stands for **exclusive creation**.
- It is used with the **open()** function to create a new file for writing.

This makes '**x**' **mode safer** than 'w' mode.

Syntax →

```
file = open("filename.txt", "x")
```

↓
name of the file
to create

↓
mode
(exclusive creation)

Important rule:

- If the file already exists, Python will **raise an FileNotFoundError instead of overwriting it**.

Using 'x' Mode

File does not exist

Python will create a new empty file. You can then write data into it.

```
with open("report.txt", "x") as f:  
    f.write("Trying to overwrite...")
```



Output:

```
report.txt  
1 This is my first report.  
2
```

File already exists

Python will NOT open the file. Instead, it will **raise an error** called **FileExistsError**.

```
with open("report.txt", "x") as f:  
    f.write("This is my first report.")
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE  
  
PS C:\Users\pralhad\Desktop\Python in Actions> & C:/Users/pralhad/AppData/Local/Programs/Python in Actions/ed.py  
Traceback (most recent call last):  
  File "c:\Users\pralhad\Desktop\Python in Actions\ed.py", line 2, in <module>  
    with open("report.txt", "x") as f:  
FileExistsError: [Errno 17] File exists: 'report.txt'  
❖ PS C:\Users\pralhad\Desktop\Python in Actions>
```

- prevents accidental overwriting of important data

Why Use 'x' Mode?

- Prevents **accidental overwriting**
- Ensures **data safety**
- Useful when creating:
 - Reports
 - Logs
 - Backups
 - Generated files

'x' vs 'w'

• 'w' → Creates file or **overwrites** if it exists

• 'x' → Creates file only if it **does not exist**

→ That is why "x" is recommended to be used

Encoding & Decoding

- What is Encoding?
- What is Decoding?
- Encoding & Decoding Combine
- What Is Character Encoding?
- Common Character Encodings
- Which UTF Encoding Are We Using in Python?
- Why Python Chose UTF-8
- Why not other UTFs?
- Industry Use Case

What is Encoding?

- Encoding is the process of **converting data from one format into another**
- so that it can be **stored, transmitted, or understood by a computer or another system.**

Encoding

```
text = "Hello, world!"  
encoded_text = text.encode("utf-8")  
print(encoded_text)  
  
# b'Hello, world!'
```

Converts string into bytes

encoding format

Indicates bytes

What is Decoding?

- Decoding is the **reverse process of encoding**.
- It **converts the encoded data back into its original**, readable form.

Decoding

```
decoded_text = encoded_text.decode("utf-8")
```

```
print(decoded_text)
```

```
# Hello, world!
```

Encoding format

Converts bytes back into text

Special Character Encoding & Decoding

Special character

```
text = "Hello, πython!"
```

```
encoded = text.encode("utf-8")
```

```
print("Encoded:", encoded)
```

```
decoded = encoded.decode("utf-8")
```

```
print("Decoded:", decoded)
```

```
'''
```

```
# Output:
```

```
→ Encoded: b'Hello, \xc0\xcf\x80ython!'
```

```
→ Decoded: Hello, πython!
```

```
'''
```

UTF-8 byte representation of π

What Is Character Encoding?

Character encoding is a rulebook that tells the computer:

- How to convert characters (like A, ₹, 😊)
- Into bytes (binary data)
- So they can **be stored, transmitted, and processed**

In short:

- Encoding : Text → Binary
- Decoding : Binary → Text

Common Character Encodings

ASCII

- One of the oldest encodings
- Supports:
 - English letters (A–Z, a–z)
 - Numbers (0–9)
 - Basic symbols
- Does not support:
 - Other languages
 - Emojis
 - Modern symbols

UTF-8

- A **Unicode encoding**
- Supports
 - **all languages,**
 - **symbols, and emojis**
- Uses **variable-length bytes**
 - Small for common characters
 - Larger only when needed
- Memory efficient
- Backward compatible with ASCII
- Widely used in:
 - Python Web, Databases, OS

UTF-16 / UTF-32

- Modern Unicode encodings
- Can represent all characters in the world
- Use fixed or larger byte sizes
- Consume more memory

UTF-8 is the **global standard for text encoding today**

Which UTF Encoding Are We Using in Python?

In modern Python (**Python 3+**), the **default text encoding is UTF-8**.

That means:

- Source code files
- Strings
- File I/O (in most environments)
- Encoding/decoding operations
- All work with **UTF-8 by default**

Without Specifying

```
text = "Hello 😊"  
encoded = text.encode()  
print(encoded)  
  
# Output : b'Hello \xf0\x9f\x98\x8a'
```

Encode without specifying

Using UTF-8

```
encoded_text = text.encode("utf-8")  
decoded_text = encoded_text.decode("utf-8")
```

text → bytes

bytes → text

Why Python Chose UTF-8

- Supports **all languages and emojis**
- Efficient memory usage
- Backward compatible with ASCII
- Internet and OS standard
- Prevents encoding errors

Important Note

- **✗ Python 2** → encoding issues, ASCII default
- **✓ Python 3** → Unicode strings + UTF-8 default

Why not other UTFs?

- **UTF-16** → Supports all characters but uses **2-4 bytes**, less efficient for English.
- **UTF-32** → Fixed 4 bytes for every character, wastes memory.
- **ASCII** → Only supports **English letters and basic symbols**, cannot handle other languages or emojis.

Industry Use Case

Problem Scenario:

- In production pipelines, **tools collect file paths from different systems and teams.**
- These paths may include non-English or special characters.
- If files are written or read without proper encoding, paths can break or tools may crash on other machines.

```
file_paths = [  
    "/mnt/projects/film01/assets/props/chair_v001.ma",  
    "/mnt/projets/film02/éléments/décor_v002.ma",  
    "/mnt/プロジェクト/映画03/アセット/小道具_v003.ma",  
    "/mnt/项目/电影04/资产/角色_v004.ma"  
]  
  
with open("file_log.txt", "w", encoding="utf-8") as log_file:  
    for path in file_paths:  
        log_file.write(path + "\n")  
  
with open("file_log.txt", "r", encoding="utf-8") as log_file:  
    paths = log_file.readlines()  
  
print("Loaded paths:")  
for path in paths:  
    print(path.strip())
```

-----> Different Language

-----> Encode in UTF - 8

Output

----->

```
Loaded paths:  
/mnt/projects/film01/assets/props/chair_v001.ma  
/mnt/projets/film02/éléments/décor_v002.ma  
/mnt/プロジェクト/映画03/アセット/小道具_v003.ma  
/mnt/项目/电影04/资产/角色_v004.ma
```

Working with Binary Files

- What Are Binary Files?
- Binary File Demo
- Reading an Image File
- Binary File Modes
- Write Binary File(wb)
- Read Binary File(rb)
- Appending to a Binary File (ab)
- Creating a Binary File Exclusively (xb)
- Read/Write Binary File(rb+)
- Text Mode vs Binary Mode

What Are Binary Files?

- Binary files **store data as raw bytes**
- The data is not human-readable text
- Files are **stored exactly as 0s and 1s**
- You **cannot read them** using a **normal text editor**

Examples of binary files:

- **Images** → `photo.jpg` `diagram.png`
- **Audio** → `song.mp3` `voice.wav`
- **Video** → `movie.mp4`
- **Executables** → `program.exe`

Any file containing **non-text data**

Binary File Demo

Suppose we want to write **"Hello, Binary World!"** into a binary file.

```
data = b"Hello, Binary World!"  
print(type(data))  
  
# <class 'bytes'>  
  
with open("sample.bin", "wb") as f:  
    f.write(data)
```

Stores raw binary data

Bytes literal

If Open on VS Code

```
sample.bin X  
1 Hello, Binary World!
```

What is stored in sample.bin?

- Bytes: 48 65 6c 6c 6f 2c 20 42 69 6e 61 72 79 20 57 6f 72 6c 64 21
- View bin data use this website: [Online binary file viewer](#)

Hexadecimal byte

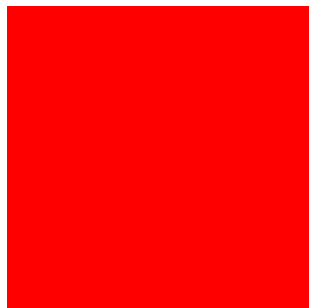
What is Binary Mode ('b')?

- **'b'** in file modes indicates **binary mode**.
 - Useful for **images, audio, video, PDFs**, or any non-text data.
- 

Reading an Image File

- Images are **pure binary data**.
- Suppose you have **image.png**.

red.png



red.png

```
with open("red.png", "rb") as f:
    content = f.read()

print(content)
print(type(content))

# <class 'bytes'>
# b'\x89PNG\r\n\x1a\n\x00\x00\x00\rIHDR\x00\x00\x00\x01'
```

Read Binary Data

indicates bytes

byte values in hexadecimal format

Binary File Modes

Mode		Description
'rb'	→	Read binary file
'wb'	→	Write binary file (overwrite if exists)
'ab'	→	Append in binary mode
'xb'	→	Create new binary file exclusively
'rb+'	→	Read/write binary file

Important Rule

In binary mode, we must work with **bytes**, not strings.

-  **Correct:** `b"Hello"`
-  **Incorrect:** `"Hello"`

Write Binary File(wb)

- Create a new binary file
- Overwrite an existing file if it already exists

```
data = b"Hello Binary World!"  
  
with open("sample.bin", "wb") as file:  
    file.write(data)  
  
print("Data written successfully.")
```

Binary literals

Write Binary

Before Update

```
0x sample.bin  
1 Hello  
2
```

After Update

```
0x sample.bin  
1 Hello Binary World!
```

Read Binary File(rb)

- Read binary data such as images or raw bytes
- **File must exist**, otherwise Python raise error.

Read Binary File

sample.bin

```
1 Hello Binary World!
```

```
with open("sample.bin", "rb") as file:  
    data = file.read()  
    print(data[:20])
```

Read Binary

Slicing

```
# Output: b'Hello, Binary World!'
```

bytes object

Read Image File

red.png



```
with open("red.png", "rb") as file:  
    data = file.read()  
    print(data[:20])
```

returns a bytes object

```
# Output: b'\x89PNG\r\n\x1a\n\x00\x00\x00\rIHDR\x00\x00\x00\x01'
```

bytes object

Appending to a Binary File (ab)

Add new binary data **without deleting existing data**

```
more_data = b" Appending some bytes."
```

```
with open("example.bin", "ab") as file:  
    file.write(more_data)
```

```
print("Data appended successfully.")
```

```
# Output: Data appended successfully.
```

Append Binary

Explanation:

- ab ensures original content remains.
- New bytes are added after existing content.

Before Appending

```
0x example.bin
```

```
1 Hello Binary World!
```

After Appending

```
0x sample.bin X
```

```
0x sample.bin
```

```
1 Hello, Binary World! Appending some bytes.
```

old content

New Content

Creating a Binary File Exclusively (xb)

- Create a **new file only if it does not exist**
- Prevents accidental overwriting
- If file exists, Python raises **FileExistsError**.

File Path

Create New File

```
with open("new_file.bin", "xb") as file:
    file.write(b"This is a new binary file!")

print("File created successfully.")

# Output: File created successfully.
```

If File Exist

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

PS C:\Users\pralhad\Desktop\Python in Actions> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39-64/Python.exe C:\Users\pralhad\Desktop\Python in Actions/demo.py
Traceback (most recent call last):
  File "c:\Users\pralhad\Desktop\Python in Actions\demo.py", line 2, in <module>
    with open("new_file.bin", "xb") as file:
FileExistsError: [Errno 17] File exists: 'new_file.bin'
PS C:\Users\pralhad\Desktop\Python in Actions>
```

If File Does not Exist

```
0x new_file.bin X
0x new_file.bin
1 This is a new binary file!
```

Read/Write Binary File(rb+)

- rb+ lets you **read and modify existing binary files in place.**
- File must exist

```
with open("sample.bin", "rb+") as file:
    original = file.read()

    print("Original content:", original)

    file.seek(0)
    file.write(b"Hi")

with open("sample.bin", "rb") as file:
    print("Modified content:", file.read())

'''
# Output:
# Original content: b'Hello, Binary World!'
# Modified content: b'Hillo, Binary World!'
'''
```

Read & write binary mode

Move cursor to file start

Update Content

Before Update

```
0x sample.bin
1  Hello Binary World!
```

After Update

```
0x sample.bin
1  Hillo Binary World!
```

Text Mode vs Binary Mode

Text Mode

- Used for **human-readable text**
- Data is handled as strings
- Python automatically applies
 - text encoding (UTF-8 by default)
- **Best for .txt, .csv, .json, .py, etc.**

```
with open("file.txt", "r") as f:
    content = f.read()

print(content)
```

- **Data type** → str

Binary Mode

- Used for **raw byte data**. Data is handled as bytes
- No encoding or decoding is applied
- Data is read exactly as stored
- **Required for images, audio, video, PDFs, EXR, etc.**

```
with open("file.bin", "rb") as f:
    content = f.read()

print(content)
```

- **Data type** → bytes

Summary

Mode	Purpose	Behavior if file exists
rb	→ Read binary	→ Error if file doesn't exist
wb	→ Write binary (overwrite)	→ Overwrites existing file
ab	→ Append binary	→ Adds data to end
xb	→ Create new binary file exclusively	→ Error if file exists
rb+	→ Read/write binary	→ File must exist

Exception Handling 2

- Revision Exception Handling
- else Block
- finally Block
- Raising Exceptions
- Validating Render Files

Revision Exception Handling

```
try:

    num = int(input("Enter a number: "))
    print("You entered:", num)

    x = int(input("\nEnter numerator: "))
    y = int(input("Enter denominator: "))

    result = x / y
    print("Result:", result)

    val = int(input("\nEnter a number for all-exception demo: "))
    print("Value entered:", val)

except ValueError:
    print("Invalid input! Please enter numbers only.")

except ZeroDivisionError:
    print("Cannot divide by zero!")

except Exception as e:
    print("Something went wrong:", e)
```

else Block

- The else block **runs only if no exception occurs in the try block.**
- Useful when you want to execute code only if everything goes well.
- **If an exception happens, the else block is skipped**

```
try:  
    x = 10 / 2  
except ZeroDivisionError:  
    print("Division by zero")  
else:  
    print("No error occurred")  
  
# No error occurred
```

Runs only if no exception occurs

When to Use else

- To separate successful execution logic from error handling
- To clearly show what should run only if no error occurs

finally Block

- The finally block **always executes** . It **runs whether an exception occurs or not**
- Commonly **used for cleanup tasks** like closing files or releasing resources.

```
f = None

try:
    f = open("data.txt", "r")
    content = f.read()
    print(content)

except FileNotFoundError:
    print("File missing")

finally:
    print("All Done !!")
    if f:
        f.close()

'''
# Output:
→ File missing
→ All Done !!
'''
```

Always executes, error or not

When to Use finally

- Closing files
- Cleaning temporary data
- Freeing system resources

Raising Exceptions

- Python allows us to **manually trigger an error using the raise keyword**
- This is useful when we want to enforce rules in our program

```
age = -5

try:
    if age < 0:
        raise ValueError("Age cannot be negative")
except ValueError as e:
    print("Error:", e)

# Error: Age cannot be negative
```

Manually raise an error

Custom Message

📌 Key Point:

- raise allows you to **manually trigger exceptions** for **invalid conditions**.

Validating Render Files

- In production tools, a **script often reads a frame range before starting a task** (render, export, cache, simulation, etc.).
- If the frame **range is invalid**, the tool must → Stop safely; Show a clear error, Clean up properly

```
start_frame = 1100
end_frame = 1000

try:
    if start_frame > end_frame:
        raise ValueError("Start frame cannot be greater than end frame")

    with open("scene.vscene", "r") as file:
        print("Scene file opened")

    except FileNotFoundError: -----> Catch missing file
        print("Scene file not found")

    except ValueError as e: -----> Catch invalid data
        print("Invalid frame range:", e)

    else: ----> Run on success
        print(f"Processing frames {start_frame} to {end_frame}")

    finally: -----> Always execute
        print("Cleanup done")
```

Output

Invalid frame range: Start frame cannot be greater than end frame
Cleanup done

Assert Keyword in Python

- What is the assert Keyword?
- Practical Demo
- Assert with Custom Error Message
- Common Usage
- assert vs if-else
- Why & Where to Use assert

What is the assert Keyword?

The **assert keyword** is a built-in Python statement used to **verify conditions during program execution**.

It checks whether a given condition is True.

- If the condition is **True** → the program continues normally
- If the condition is **False** → Python raises an AssertionError and stops execution

Basic Syntax →

```
assert condition, "Optional error message"
```

expression that must be *True*

explains what went wrong (recommended)

The main purpose of assert is **debugging and validation during development**.

Practical Demo

Let's say we want to ensure a **number is positive**:

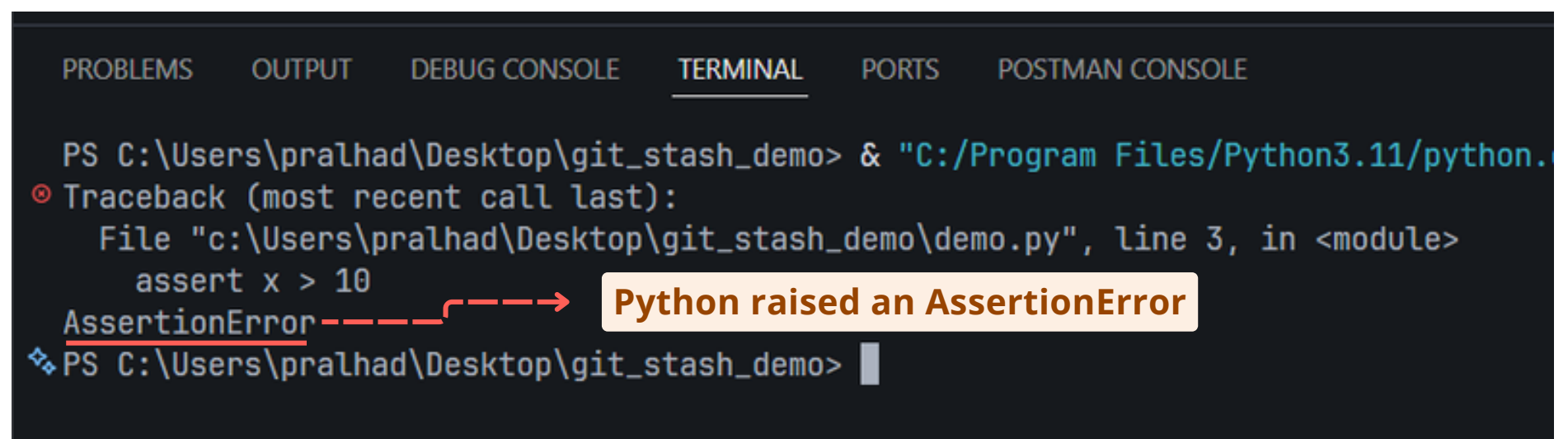
```
x = 10
assert x > 0
print("x is positive")
```

Output:

```
x is positive
```

If we change x to a **negative** number:

```
x = -5
assert x > 0
print("x is positive")
```



The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, PORTS, and POSTMAN CONSOLE. The terminal output shows a command prompt running a Python script. The script contains an assertion that fails because x is -5. The terminal displays a traceback indicating an AssertionError at line 3 of the script. A red arrow points from the 'AssertionError' text in the traceback to a yellow box containing the text 'Python raised an AssertionError'.

```
PS C:\Users\pralhad\Desktop\git_stash_demo> & "C:/Program Files/Python3.11/python.  
Traceback (most recent call last):  
  File "c:\Users\pralhad\Desktop\git_stash_demo\demo.py", line 3, in <module>  
    assert x > 10  
AssertionError-----> Python raised an AssertionError  
PS C:\Users\pralhad\Desktop\git_stash_demo> █
```

The program stopped immediately

Understanding Execution Flow of assert

```
print("Start")  
  
assert 5 > 10, "Condition failed"  
  
print("End") -----> never print
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE  
  
PS C:\Users\pralhad\Desktop\git_stash_demo> & "C:/Program Files/Python3.11/python.  
⦿ Start  
Traceback (most recent call last):  
  File "c:\Users\pralhad\Desktop\git_stash_demo\demo.py", line 3, in <module>  
    assert 5 > 10  
AssertionError  
❖ PS C:\Users\pralhad\Desktop\git_stash_demo>
```

Python evaluates the condition after **assert**

- **If the condition is True**
 - Nothing happens
 - Execution continues
- **If the condition is False**
 - AssertionError is raised
 - Program execution stops (unless handled)

Assert with Custom Error Message

We can attach a meaningful message to explain **why** the assertion failed.

```
x = -10
assert x > 0, "x must be a positive number"
```

↑
Custom message

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

PS C:\Users\pralhad\Desktop\git_stash_demo> & "C:/Program Files/Python3.11/python.exe"
Traceback (most recent call last):
  File "c:\Users\pralhad\Desktop\git_stash_demo\demo.py", line 2, in <module>
    assert x > 0, "x must be a positive number"
AssertionError: x must be a positive number
PS C:\Users\pralhad\Desktop\git_stash_demo>
```

This is extremely **useful** when debugging larger scripts.

Common Usage

Validating Function Preconditions

Use case → Ensuring a function receives valid data **from inside the system**, not from users.

```
def calculate_frame_time(fps):  
    assert fps > 0, "FPS must be greater than 0"  
    return 1 / fps  
  
print(calculate_frame_time(24))  
  
print(calculate_frame_time(0))
```

Output (assert passes):

```
0.041666666666666664
```

Output (assert fails):

```
AssertionError: FPS must be greater than 0
```

Used in pipelines, engines, internal tools.

Verifying Pipeline Assumptions

Use case → Ensuring required data is present before processing.

```
def process_exr_metadata(metadata):  
    assert "width" in metadata, "EXR width missing"  
    assert "height" in metadata, "EXR height missing"  
  
    print("Processing EXR:", metadata["width"], "x", metadata["height"])  
  
process_exr_metadata({"width": 1920, "height": 1080})  
  
process_exr_metadata({"width": 1920})
```

Output (assert passes):

```
Processing EXR: 1920 x 1080
```

Output (assert fails):

```
AssertionError: EXR height missing
```

This prevents broken data from moving deeper into the pipeline.

Catching Logic Errors Early

Use case → Ensuring logic invariants stay true during development.

```
def normalize_value(value):  
    result = value / 10  
    assert 0 <= result <= 1, "Normalized value out of range"  
    return result  
  
print(normalize_value(5))  
print(normalize_value(50))
```

Output (assert passes):

0.5

Output (assert fails):

AssertionError: Normalized value out of range

Helps catch **wrong math or broken logic** immediately.

assert vs if-else

GamutX

using if-else

```
def validate_exr_channels_if(channels):  
    if "R" not in channels:  
        print("R channel missing")  
        return  
    if "G" not in channels:  
        print("G channel missing")  
        return  
    if "B" not in channels:  
        print("B channel missing")  
        return  
  
    print("EXR channel validation passed.")  
  
validate_exr_channels_if({"R": 1.0, "G": 1.0, "B": 1.0})  
validate_exr_channels_if({"R": 1.0, "B": 1.0})  
  
# Output:  
# EXR channel validation passed.  
# G channel missing
```

Conditional
validation check

Why this is a problem

- Pipeline **continues running**
- Broken EXR is only printed, not stopped
- Bug appears later and is harder to debug

Using assert

```
def validate_exr_channels_assert(channels):  
    assert "R" in channels, "R channel missing"  
    assert "G" in channels, "G channel missing"  
    assert "B" in channels, "B channel missing"  
  
    print("EXR channel validation passed.")  
  
validate_exr_channels_assert({"R": 1.0, "G": 1.0, "B": 1.0})  
validate_exr_channels_assert({"R": 1.0, "B": 1.0})  
  
# Output:  
# EXR channel validation passed.  
# AssertionError: G channel missing
```

Enforces required
condition

Why this is correct

- Missing RGB = **pipeline bug**
- Execution **stops immediately**
- Developer knows **exactly what broke**
- No corrupted data moves forward

assert fails fast; if-else only reports errors

Note

- **if-else** → Handles **expected** runtime situations.
 - **assert** → Checks things that **should NEVER** be wrong if the pipeline is correct.
- 

Why & Where to Use assert

During Development

- During development, developers make **assumptions** about data coming into functions.
- assert is used to **validate those assumptions early**, so incorrect data never flows deeper into the pipeline.

Without assert

```
def process_exr_resolution(width, height):  
    pixels = width * height  
    print(f"Processing EXR with {pixels} pixels")  
  
process_exr_resolution(1920, 1080)  
process_exr_resolution(0, 1080)
```

Output:

```
Processing EXR with 2073600 pixels  
Processing EXR with 0 pixels
```

Problem:

- Code runs, No error & No warning
- Render silently does nothing

With assert

```
def process_exr_resolution(width, height):  
    assert width > 0, "EXR width must be greater than 0"  
    assert height > 0, "EXR height must be greater than 0"  
  
    pixels = width * height  
    print(f"Processing EXR with {pixels} pixels")  
  
process_exr_resolution(1920, 1080)
```

Output:

```
Processing EXR with 2073600 pixels  
AssertionError: EXR width must be greater than 0
```

- Immediate feedback
- Error points to the **real cause**

Simulating Tests

- Assertions are also used to **verify expected behavior** of functions.
- This simulates testing logic and ensures that changes do not break existing functionality.

Without assert

```
def calculate_exr_bit_depth(exr_info):  
    return exr_info["bit_depth"]  
  
print(calculate_exr_bit_depth({"bit_depth": 16}))  
print(calculate_exr_bit_depth({"bit_depth": 32}))
```

Output:

```
16  
32
```

- The function executes, but **no correctness is verified**.
- If the logic changes incorrectly
 - the pipeline will not notice.

With assert

```
def calculate_exr_bit_depth(exr_info):  
    return exr_info["bit_depth"]  
  
assert calculate_exr_bit_depth({"bit_depth": 16}) == 16  
assert calculate_exr_bit_depth({"bit_depth": 32}) == 32  
  
print("All EXR bit-depth tests passed successfully.")  
  
# Output : All EXR bit-depth tests passed successfully.
```

If someone accidentally breaks this function:

- assert catches it instantly
- Pipeline doesn't ship broken EXRs downstream

Key Takeaways

- assert **validates developer assumptions**
 - Helps detect errors **early in development**
 - Prevents silent pipeline corruption
 - Acts as **lightweight testing**
 - Forms the foundation of testing frameworks
- 

Thank You



Bitwise Operators

- What is a Bitwise Operators?
- What Are Bits & Bytes?
- Understanding Binary Numbers
- Types of Bitwise Operators
- AND (&)
- OR (|)
- XOR (^)
- NOT (~)
- Left Shift (<<)
- Right Shift (>>)

What is a Bitwise Operators?

- Bitwise operators are **special symbols or keywords**
- operate on **integer data at the level of individual bits (0s and 1s)**.
- They manipulate the binary representation of a number directly, rather than the number as a single value.

What Are Bits & Bytes?

Bit

- A bit is the **smallest unit of data in a computer**.
- It can be only:
 - 0 → OFF
 - 1 → ON

Byte

1 byte = 8 bits

Example: `01001011 → 8 bits = 1 byte`

Think of bits as tiny switches:

- 0 = OFF
- 1 = ON

Understanding Binary Numbers

Storing the Number 5 in Binary

Step 1: Convert Decimal to Binary

Decimal 5 → Binary 0101

Step 2: Assign Place Values (Powers of 2)

Binary place values increase from **right to left**:

Bit Position → 2^3 2^2 2^1 2^0

Value → 8 4 2 1

Align bits with place values:

Bit:	0	1	0	1
Place:	8	4	2	1

Step 3: Multiply Bits by Place Values

0	×	8	=	0
1	×	4	=	4
0	×	2	=	0
1	×	1	=	1

Step 4: Add the Results

0 + 4 + 0 + 1 = 5 ✓

This confirms that:

Binary 0101 represents Decimal 5

Types of Bitwise Operators

AND (&)

OR (|)

XOR (^)

NOT (~)

Left Shift (<<)

Right Shift (>>)

AND (&)

Rule: Result is **1** only if both bits are **1**, otherwise **0**.

Example:

a = 5 → binary 0101
b = 3 → binary 0011

```
result = a & b  
print(result) # Output: 1
```

Truth Table

A		B		A & B
0	→	0	→	0
1	→	0	→	0
0	→	1	→	0
1	→	1	→	1

OR (|)

Rule: Result is 1 if **at least one bit** is 1.

Example:

a = 5 → binary 0101

b = 3 → binary 0011

result = a | b → bitwise OR compares each bit:
print(result) # Output: 7

Truth Table

A		B		A B
0	→	0	→	0
1	→	0	→	1
0	→	1	→	1
1	→	1	→	1

XOR (^)

Rule: Result is **1** if bits are different, otherwise **0**.

Example:

```
a = 5 → binary 0101
b = 3 → binary 0011

result = a ^ b → bitwise XOR compares each bit:
print(result) # Output: 6
```

Truth Table

A		B		A ^ B
0	→	0	→	0
0	→	1	→	1
1	→	0	→	1
1	→	1	→	0

NOT (~)

- **Rule:** Flips all bits (0 → 1, 1 → 0).
- In Python, it also changes the sign: $\sim x = -(x+1)$
- Works on **single operand**, so no two-input truth table.

Example:

```
a = 5 -----> 0101
```

```
result = ~a -----> Bitwise NOT flips every bit:  
print(result) # Output: -6
```

Truth Table

A		~A
0	→	1
1	→	0

Left Shift (<<)

- **Rule:** Shift bits to the left, adding zeros on the right.
- Equivalent to multiplying by 2^n where n is the number of shifts.
- **Idea:** Move all bits to the **left**. Empty spots on the right are filled with 0.

Example:

a = 5 $\longrightarrow$ 0101

```
result = a << 1  
print(result) # Output: 10
```

0101 << 1 $\rightarrow$ 1010 $\rightarrow$ 10

- Binary 1010 = 10 in decimal
 - So $5 << 1 = 10$
- Shift left by 2 ($a << 2$):

0101 << 1 $\rightarrow$ 1010 $\rightarrow$ 10

- Binary 10100 = 20 in decimal
 - So $5 << 2 = 20$

Each left shift multiplies the number by 2 for every shift.

Right Shift (>>)

- **Rule:** Shift bits to the right, discarding bits on the right.
- Equivalent to floor dividing by 2^n .
- **Idea:** Move all bits to the **right**. Bits that “fall off” are removed.

Example:

a = 5 $\longrightarrow$ 0101

```
result = a >> 1  
print(result) # Output: 2
```

0101 >> 1 $\rightarrow$ 0010 $\rightarrow$ 2

- Binary 0010 = 2 in decimal
 - ☒ So $5 \gg 1 = 2$
- Shift right by 2 ($a \gg 2$):

0101 >> 2 $\rightarrow$ 0001

- Binary 0001 = 1 in decimal
 - So $5 \gg 2 = 1$

Each right shift divides the number by 2 (ignoring remainders).